Module 3

Structural Testing

Overview

Another type of **software testing** technique is **Structural testing**, which is used to test the internal design of the software or structure of the coding for the particular software. In this testing, the development team members are included in the testing team to execute the software's internal design. The working of structural testing is opposite to **Behavioral testing**.

In other words, we can say that structural testing tests the different features of an application based on its types. Structural testing is also known as white-box testing, **glass box testing**, and **clear-box testing**. Developers mostly implement it to identify the issue and fix them quickly

The structural testing process requires an in-depth knowledge of the programming language and is opposite to Functional Testing. The knowledge of the code's internal executions and how the software is implemented is a necessity for the test engineer to implement the structural testing.

Throughout the structural testing, the test engineer intends on how the software performs, and it can be used at all levels of testing.

Statement Coverage

It is one type of white box testing technique that ensures that all the statements of the source code are executed at least once. It covers all the paths, lines, and statements of a source code. It is used to design test box cases where it will find out the total number of executed statements out of the total statements present in the code.

Formula:

Statement coverage = (Number of executed statements / Total number of statements in source code) * 100

Example 1:

```
Read A
Read B
if A > B
   Print "A is greater than B"
else
```

```
   Print "B is greater than A"
endif
```

Case 1:

If A = 7, B= 3

***No of statements Executed**= 5*

***Total statements**= 7*

***Statement coverage**= 5 / 7 * 100*

= 71.00 %

Case 2:

If A = 4, B= 8

***No of statements Executed**= 6*

***Total statements**= 7*

***Statement coverage**= 6 / 7 * 100*

= 85.20 %

Example 2:

```
print (int a, int b)
{
   int sum = a + b;
   if (sum > 0)
     print ("Result is positive")
  else
    print ("Result is negative")
}
```

Case 1:

If A = 4, B= 8

***No of statements Executed**= 6*

***Total statements**= 8*

***Statement coverage**= 6 / 8 * 100*

= 75.00 %

Case 2:

If A = 4, B= -8

***No of statements Executed**= 7*

***Total statements**= 8*

***Statement coverage**= 7 / 8 * 100*

= 87.50 %

In the internal code structure, there are loops, arrays, methods, exceptions, and control statements. Some code would be executed based on input while some may not. Statement coverage will execute all possible paths and statements of the code

Statement coverage covers:

- Dead code.
- Unused statements.
- Unused branches.
- Missing statements.

Why is statement coverage used?

- To check the quality of the code.
- To determine the flow of different paths of the program.
- Check whether the source code expected to perform is valid or not.
- tests the software's internal coding and infrastructure.

Drawback of Statement Coverage:

- Cannot check the false condition.
- Different input values to check all the conditions.
- More than one test case may be required to cover all the paths with a coverage of 100%.

Branch coverage

Branch coverage in unit testing is a metric that measures the percentage of branches (decision points) in the source code that have been executed during the testing process. It indicates how well the test cases navigate through different possible outcomes of conditional statements, helping evaluate the thoroughness of testing. A high branch coverage means that most decision paths in the code have been tested, increasing the likelihood of detecting potential defects.

1. Achieving comprehensive branch coverage is important for ensuring the reliability and effectiveness of unit tests in identifying and addressing code issues.
2. This metric helps identify areas of code that may not have been adequately tested, increasing the likelihood of detecting potential defects and enhancing the overall reliability and quality of the software.

Formula for Branch Coverage:

Branch Coverage = (Total Number of Branches) ÷ (Number of Executed Branches) ✕ 100%

Example of Branch Coverage in Unit Testing

Consider the following simple function that determines whether a person is eligible to vote based on their age. The function has two conditional statements (if and else) and one unconditional print statement. Branch coverage in unit testing for this function aims to cover all possible branches, including both conditional and unconditional statements.

```
checkAge( int age) {
if (age >= 18)
print("Eligible to Vote")
else
print("Not Eligible to Vote")
print("Print the Age: %d" + age)
}
```

Branch Coverage Test Cases

1. **Test Case 1 (True Positive):** Create a test case to cover the if condition by providing an age greater than or equal to 18. This scenario represents a True Positive, where the individual is eligible to vote. This test case covers the if part of the function.
2. **Test Case 2 (False Positive):** Generate a test case to cover the other condition by providing an age less than 18. This represents a False Positive scenario, where the individual is not eligible to vote. This test case covers the other part of the function.
3. **Test Case 3 (Print Statement):** Design a test case to cover the print statement by providing mock data for age and verifying that the age is printed correctly. This test case ensures coverage of the common print statement executed in the function.

Condition coverage:-

The software condition coverage testing is a part of the white box testing. It is used by the developers to design white box test cases. It is primarily focused on covering all the conditions in the program source code. It is also referred to as predicate coverage testing.

In the software condition coverage testing every Boolean expression described in the conditions expression is evaluated to both true and false outcomes. As a result, it ensures that both the branches in a decision statement are tested. In case, a decision statement comprises various

conditions namely OR, and AND, the condition coverage testing confirms that all the various combinations of the conditions are included in the test cases

Formula to Calculate the Software Condition Coverage Percentage

The software condition coverage can be calculated by dividing the total count of conditions executed with the total count of conditions in the source code, and then multiplied by hundred.

Condition Coverage = (Total count of conditions executed / Total count of conditions in the source code) * 100

Advantages of Software Condition Coverage Testing

The advantages of the software condition coverage testing are listed below −

- The software condition coverage testing guarantees that all the conditions in the code are checked at least once.
- The software condition coverage testing detects defects from the early stages of the software development life cycle(SDLC).
- The condition coverage testing improves the quality, maintainability, and reliability of the software.
- The software condition coverage testing helps in faster troubleshooting of errors in the code.
- The condition coverage testing gives higher confidence, and trust on the code developed for the software.

Example

Let us take an example of the below code snippet to determine the count of the condition coverage.

```
Input X, Y, Z, and W
IF (X == 0 || Y == 0)
  THEN PRINT 100
  ELSE IF (Z == 0 && W == 0)
    THEN PRINT 200
  END IF
END
```

Let us now calculate the condition coverage with the first test case, with the inputs X = 0, Y = 0, Z = 0, W = 0. With these values, the first condition of the OR operator with the expression X == 0, holds true. Since the result of the left side of the OR is already true, so the right side of the expression, Y == 0 of it, will be skipped from evaluation.

So, the condition Y == 0 remained unchecked. Then, the statement 3 will be executed resulting in printing 100 and the condition IF (Z == 0 && W == 0) in the ELSE part(at the line 4) will not be executed. Hence, out of a total four conditions, only one of them got executed. The total condition coverage as per formula:

Condition Coverage = (Total count of conditions executed / Total count of conditions in the source code) * 100

Condition Coverage = (1 / 4) * 100 = 25 %.

Let us now calculate the condition coverage with the second test case, with the inputs X = 1, Y = 0, Z = 0, W = 0. With these values, the first condition having the OR operator with the expression X == 1, holds false.

As the result of the left side of the OR is false, so the right side expression Y == 0 of it, will be evaluated. So this time, the condition Y == 0 is checked. Then, the statement 3 will be executed resulting in printing 100 and the condition IF (Z == 0 && W == 0) in the ELSE part(at the line 4) will remain unchecked. Hence, out of a total four conditions, two of them got executed. The total condition coverage as per formula

Condition Coverage = (Total count of conditions executed / Total count of conditions in the source code) * 100

Condition Coverage = (2 / 4) * 100 = 50 %.

Let us now calculate the condition coverage with the third test case, with the inputs X = 1, Y = 1, Z = 0, W = 0. With these values, the first condition having the OR operator with the expression X == 1, holds false.

As the left side of the OR is false, so the right side expression Y == 1 of it, will be evaluated. Since both sides of the OR operator result in false, so this time, the condition IF (Z == 0 && W == 0) in the ELSE part(at the line 4) will be executed.

The first condition having the AND operator with the expression Z == 0, holds true. Though the left side of the AND operator is true, the right side expression W == 0 will also need to be evaluated. Since both of them are true, then, the statement 5 will be executed resulting in printing 200. Hence, out of total four conditions, four of them got executed this time. The total condition coverage as per formula

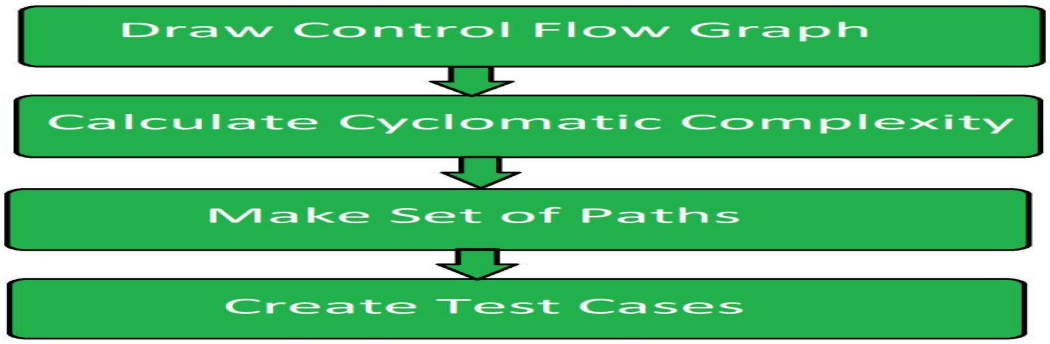
Condition Coverage = (Total count of conditions executed / Total count of conditions in the source code) * 100

Condition Coverage = (4 / 4) * 100 = 100 %.

Path based testing

Path Testing is a method that is used to design the test cases. In the path testing method, the control flow graph of a program is designed to find a set of linearly independent paths of execution. In this method, Cyclomatic Complexity is used to determine the number of linearly independent paths and then test cases are generated for each path. It gives complete branch coverage but achieves that without covering all possible paths of the control flow graph. McCabe's Cyclomatic Complexity is used in path testing. It is a structural testing method that uses the source code of a program to find every possible executable path.

Path Testing Process



Path Testing

Control Flow Graph:

Draw the corresponding control flow graph of the program in which all the executable paths are to be discovered.

- **Cyclomatic Complexity:**
  After the generation of the control flow graph, calculate the cyclomatic complexity of the program using the following formula.

  Cyclomatic Complexity = E - N + 2P

  Where,
  E = Number of edges in the control flow graph
  N = Number of vertices in the control flow graph
  P = Program factor

- **Make Set:**
  Make a set of all the paths according to the control flow graph and calculate cyclomatic complexity. The cardinality of the set is equal to the calculated cyclomatic complexity.
- **Create Test Cases:**
  Create a test case for each path of the set obtained in the above step.

Path Testing Techniques

- **Control Flow Graph:**
  The program is converted into a control flow graph by representing the code into nodes and edges.
- **Decision to Decision path:**
  The control flow graph can be broken into various Decision to Decision paths and then collapsed into individual nodes.
- **Independent paths:**
  An Independent path is a path through a Decision to Decision path graph that cannot be reproduced from other paths by other methods.

Advantages of Path Testing

1. The path testing method reduces the redundant tests.
2. Path testing focuses on the logic of the programs.
3. Path testing is used in test case design.

Disadvantages of Path Testing

1. A tester needs to have a good understanding of programming knowledge or code knowledge to execute the tests.
2. The test case increases when the code complexity is increased.
3. It will be difficult to create a test path if the application has a high complexity of code.

4. Some test paths may skip some of the conditions in the code. It may not cover some conditions or scenarios if there is an error in the specific paths.

Data flow testing

Data flow testing focuses on the points at which variables **receive** values and the points at which these values are **used (or referenced).** It **detects improper use of data values** (data flow anomalies) due to coding errors.

Rapps and Weyukers Motivation*: " it is our belief that, just as one would not feel confident about a program without executing every statement in it as part of some test, **one should not feel confident** about a program without having seen the effect of Using the value produced by each and every computation.

Early data flow analyses often centered on a set of faults that are known as define/reference anomalies.

- A variable that is defined but never used (referenced)
- A variable that is used but never defined
- A variable that is defined twice before it is used

- **Data flow testing**

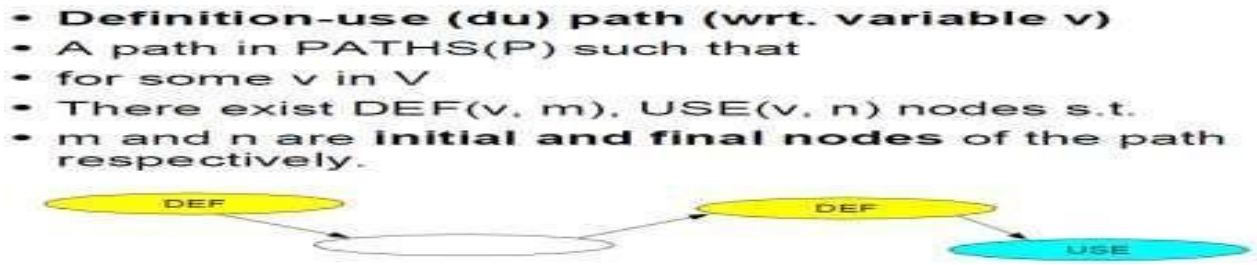
1. Define / Use Testing
2. Slice-Based Testing

DEFINE/USE TESTING

The following refers to program P that has program graph G (P) and the set of program variables V. In a program graph statement fragments are nodes and edges represents node sequence .G (P) has single entry and single exit node. We also disallow edges from node to itself. The set of all paths in P is PATHS (P).

Defining and Usage Nodes

- **Defining node (e.g. input x, v = 2, etc.):** *DEF(v, n):* Node n in G(P) is a defining node of var v in V iff value of v is **defined** at n.
- **Usage node (e.g. output x, a = 2+v, etc.):** *USE(v, n):* Node n in G(P) is a usage node of var v in V iff value of v is **used** at n.



A usage node USE(v,n) is predicate use denoted as P-case , if statement n is predicate statement (example if a<2)

If statement n is computation statement is denoted as C-case (example C=C+2)

Example :-(Commission problem)

1. program commission(INPUT, OUTPUT)

2. Dim lock , stock , barrels as Integer

3. Dim lockprice , stockprice , barrelprice As Real

4. Dim totalLocks , totalStocks , totalBarrels As Integer

5. Dim lockSales, stocksales ,barrelsSales As Real

6. Dim sales , commission As Real

7. lockprice=45.0

8 stockprice= 30.0Software Testing

Page 26

9. barrelprice = 25.0

10. totallocks=0

11. totalstocks=0

12. totalbarrels=0

13 .input(locks)

14.Whle not (locks= -1)

15.Input (stock,barrel)

16.Totallocks =total locks +locks

17. Totalstocks =total stocks +stocks

18. Totalbarrels = totalbarrels +barrels

19. input (locks)

20.End While

21. output (“Locks sold”, total locks)

22. output (“Stocks sold”, total stocks)

23. output (“Barrels sold”, totalbarrels)

24.locksales= lockprice *totallocks

25.stocksales= stockprice *totalstocks

26.barrelsales= barrelprice *totalbarrels

27 sales= locksales + stocksales+barrelsales

28. output(“Totalsales”, sales)

29. if (sales > 1800.0)

30. then

31. commission = 0.10 * 1000

32. commission =commission +0.15 *800.0

33.commission = commission +0.20 *(sales >1000)

34. Else if (sales >1000)

35.Then

36. commission = 0.10 * 1000.0

37. commission = commission +0.15 *(sales-1000.0)

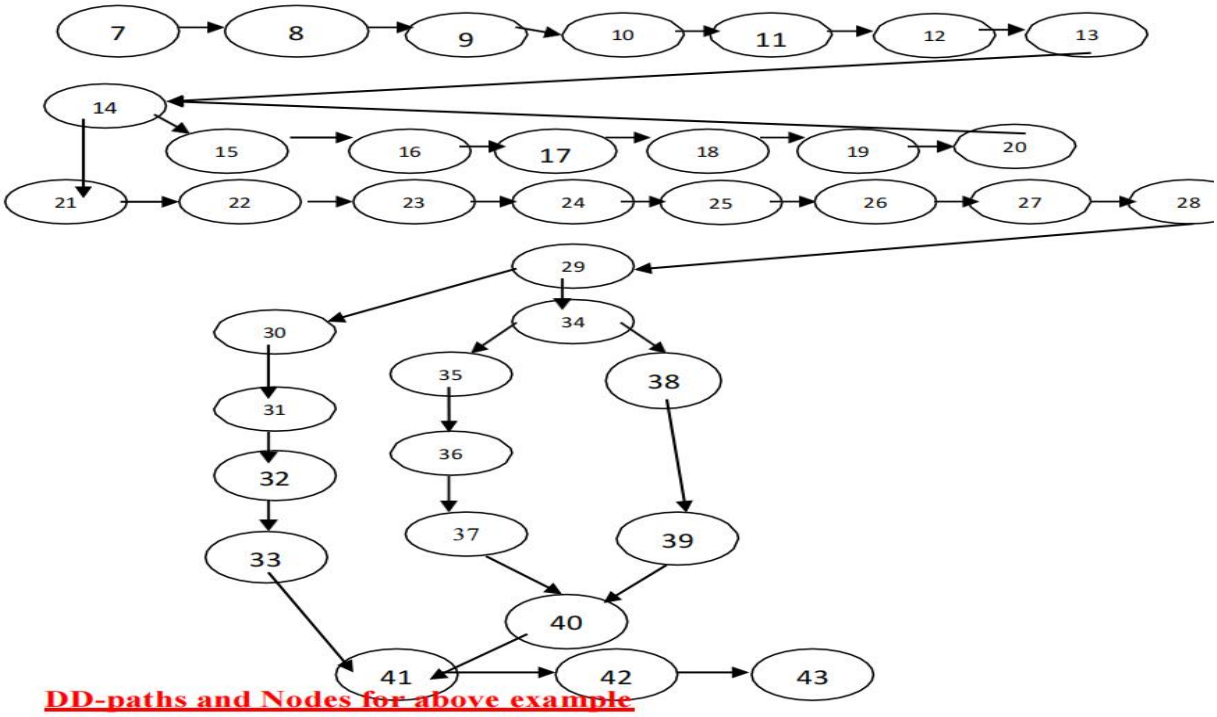
38. Else

39. commission =0.10 * sales

40. End IfSoftware Testing

41. End If

42. output (“commission is $”, commission)



DD-paths and Nodes for above example

DD-paths	Nodes
A	7,8,9,10,11,12,13,
B	14
C	15,16,17,18,19,20
D	21,22,23,24,25,26,27, 28
E	29
F	30,31,32,33
G	34
H	35,36,37
I	38,39
J	40
K	41,42,42

du-Paths for Stocks:

First, let us look at a simple path:the du-path or the variable stocks. We have DEF(stocks,15) and USE(stocks,17),so the path<15,17> is adu-path with respect to stocks. No other is defining nodes are used for stocks; therefore, this path also definition clear.

du-Paths for Locks:

Two defining and two usage nodes make the locks variable more interesting: we have DEF(locks,13), DEF(locks,19),USE(locks,14),and USE(locks,16). These yield four

du-paths:

P1=<13,14>

P1=<13,14,15,16>

P1=<19,20,14>

P1=<19,20,14,15,16>

Du-paths p1 and p2 refer to the priming Value of locks, which is read at node 13: locks has a predicate use in the while statement(node 14), and if the condition is true (as in path p2), a computation use at statement 16.The other two du-paths start near the end of the while loop and occur when the loop repeats.

du-Paths for totalLocks:

The du-paths for totallocks will be lead us to typical test cases for computations. With two defining nodes(DEF(toalLocks, 10) and DEF(totallocks, 16)) and three usage nodes (USE(totalLocks,16),USE(totalLocks,21), USE(totalLocks,24)), We might expect six du-paths. Let us take a closer look. Path p5=<10,11,12,13,14,15,16> is a du-path in which the initial value of totalLocks (0) has computation use.

du-Paths for Sales:-

Only one defining node is used for sales; therefore, all the du-paths with respect to sales must be definition-clear. They are interesting because they illustrate predicate and computation uses. The First 3 du-paths are easy:

P10=<27,28>

P11=<27,28,29>

Slice based testing

Data flow testing focuses on the points at which variables **receive** values and the points at which these values are **used (or referenced).** It **detects improper use of data values** (data flow anomalies) due to coding errors.

Rapps and Weyukers Motivation*: " it is our belief that, just as one would not feel confident about a program without executing every statement in it as part of some test, **one should not feel confident** about a program without having seen the effect of Using the value produced by each and every computation.

Slice-Based Testing

The following refers to program P that has program graph G (P) and the set of program variables V. In a program graph statement fragments are nodes and edges represents node sequence .G (P) has single entry and single exit node. We also disallow edges from node to itself. The set of all paths in P is PATHS (P)

Definition:- Given a program P and a set V of variables in P, a slice on the variable set V at statement n, written S(V, n), is the set of all statements in P prior to node n that contribute to the values of variables in V at node n. Listing elements of a slice S(V, n) will be cumbersome because the elements are program statement fragments. It is much simpler to list fragment numbers in P(G).

USE TYPES	DEF TYPES
P-use - Used in a predicate stmt	I-def -Defined by input
C-use - Used in computation	A-def -Defined by assignment
O-use -Used for output	
L-use - used for location (pointers)	
I-use Iteration (Internal counters, loop indices)	

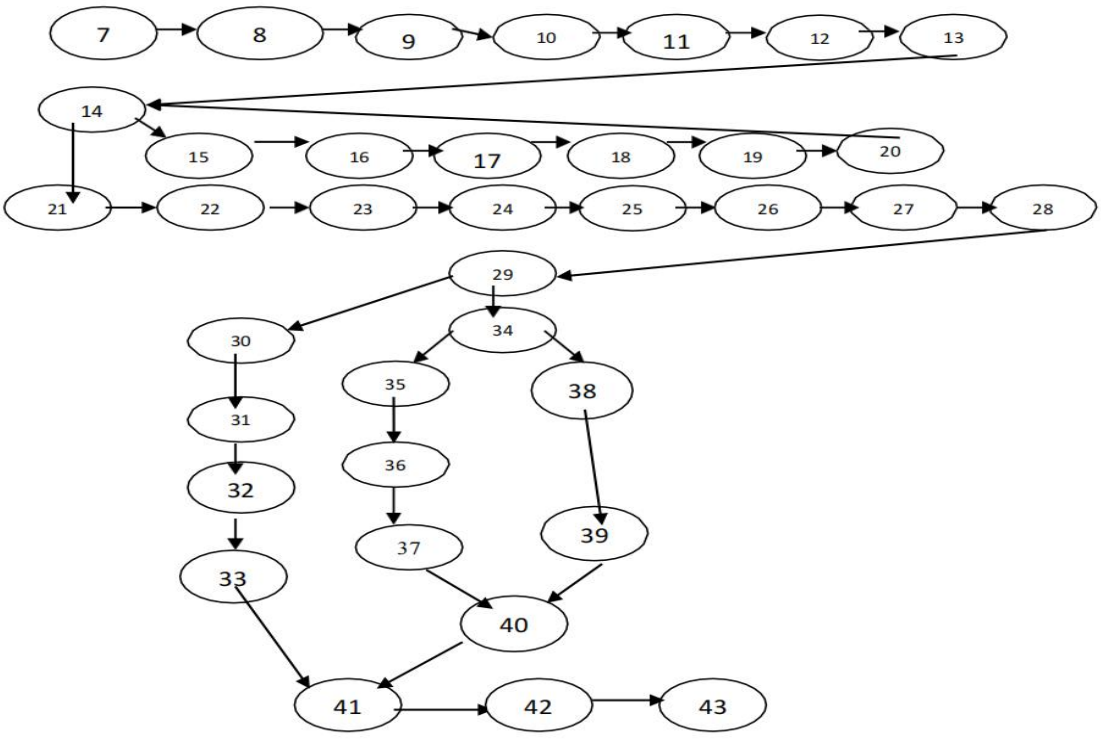
Example :- The commission problem is used here because it contains interesting dataflow properties , and these are not present in the triangle problem(or in next date function).Follow these examples while looking at the source code for the commission problem that we used to analyse in terms of define/use paths.

(Commission problem)

```
1. program commission(INPUT, OUTPUT)
2. Dim lock , stock , barrels as Integer
3. Dim lockprice , stockprice , barrelprice As Real
4. Dim totalLocks , totalStocks , totalBarrels As Integer
5. Dim lockSales, stocksales ,barrelsSales As Real
6. Dim sales , commission As Real
7. lockprice=45.0
8 stockprice= 30.0
9. barrelprice = 25.0
10. totallocks=0Software Testing
11. totalstocks=0
12. totalbarrels=0
13 .input(locks)
14.Whle not (locks= -1)
15.Input (stock,barrel)
16.Totallocks =total locks +locks
17. Totalstocks =total stocks +stocks
18. Totalbarrels = totalbarrels +barrels
19. input (locks)
20.End While
21. output ( “Locks sold”, total locks)
22. output (“Stocks sold”, total stocks)
23. output (“Barrels sold”, totalbarrels)
24.locksales= lockprice *totallocks
```

25.stocksales= stockprice *totalstocks
26.barrelsales= barrelprice *totalbarrels
27 sales= locksales + stocksales+barrelsales
28. output(“Totalsales”, sales)
29. if (sales > 1800.0)
30. then
31. commission = 0.10 * 1000
32. commission =commission +0.15 *800.0
33.commission = commission +0.20 *(sales >1000)
34. Else if (sales >1000)
35.Then
36. commission = 0.10 * 1000.0
37. commission = commission +0.15 *(sales-1000.0)
38. Else
39. commission =0.10 * salesSoftware Testing
40. End If
41. End If
42. output (“commission is $”, commission)
43. End commission

Program graph of the the commission problem



Slices on the locks variable show why it is potentially fault-prone.it has a P-use at node 14 and a C-use at node 16 and has two definitions, the I-defs at nodes 13 and 19.

S1:S (locks,13)={13}
S2:S(locks, 14)={13,14,19,20}
S3:S (locks,16)={13,14,19,20}
S4:S(locks,19)={19}

The Slices for stocks and barrels are boring. They are short, definition-clear paths contained Entirely within a loop, so they are not affected by iterations of the loop. (Think of the loop body As a DD-Path.)

S5:S(stocks,15)={13,14,15,19,20}
S6:S(stocks,17)={13,14,15,19,20}
S7:S(barrels,15)={13,14,15,19,20}
S8:S(barrels,18)={13,14,15,19,20}

The next three slices illustrates how repetation appears in slices. Node 10 is an A-def for totalLocks And node 16 contains both an A-def and a C-use. The remaining nodes in S10 (13, 14,19 and 20) pertain to the While loop controlled by locks. Slice S10 and S11 are equal because nodes 21 and 24 are an O-use and a C-use of totalLocks respectively.

S9:S(totalLocks,10)={10}
S10:S(totalLocks,16)={10,13,14,16,19,20}
S11:S(totalLocks,21)={10,13,14,16,19,20}

The slices on totalStocks and totalBarrels are quite similar. They are initialized by Adefs at nodes 11 and 12 and then are defined by A-defs at nodes 17 and 18. Again, the remaining nodes (13,14,19 and 20) pertains to the While loop controlled by locks.

S12:S(totalLocks,11)={11}
S13:S(totalLocks,17)={11,13,14,15,17,19,20}
S14:S(totalLocks,22)={11,13,14,15,17,19,20}
S15:S(totalBarrels,12)={12}
S16:S(totalBarrels,18)={12,13,14,15,16,19,20}
S17:S(totalBarrels,23)={12,13,14,15,18,19,20}

Test Execution

Test Execution is the process of executing the tests written by the tester to check whether the developed code or functions or modules are providing the expected result as per the client requirement or business requirement. Test Execution comes under one of the phases of the Software Testing Life Cycle (STLC).

In the test execution process, the tester will usually write or execute a certain number of test cases, and test scripts or do automated testing. If it creates any errors then it will be informed to the respective development team to correct the issues in the code. If the text execution process shows successful results then it will be ready for the deployment phase after the proper setup for the deployment environment.

Importance of Test Execution:

- **The project runs efficiently:** Test execution ensures that the project runs smoothly and efficiently.
- **Application competency:** It also helps to make sure the application's competency in the global market.
- **Requirements are correctly collected:** Test executions make sure that the requirements are collected correctly and incorporated correctly in design and architecture.
- **Application built in accordance with requirements:** It also checks whether the software application is built in accordance with the requirements or not.

Activities for Test Execution

The following are the 5 main activities that should be carried out during the test execution.

1. **Defect Finding and Reporting:** Defect finding is the process of identifying the bugs or errors raised while executing the test cases on the developed code or modules. If any error appears or any of the test cases failed then it will be recorded and the same will be reported to the respective development team. Sometimes, during the user acceptance testing also end users may find the error and report it to the team. All the recorded details will be reported to the respective team and they will work on the recorded errors or bugs.
2. **Defect Mapping:** After the error has been detected and reported to the development team, the development team will work on those errors and fix them as per the requirement. Once the development team has done its job, the tester team will again map the test cases or test scripts to that developed module or code to run the entire tests to ensure the correct output.
3. **Re-Testing:** From the name itself, we can easily understand that Re-Testing is the process of testing the modules or entire product again to ensure the smooth release of the module or product. In some cases, the new module or functionality will be developed after the product

release. In this case, all the modules will be re-tested for a smooth release. So that it cannot cause any other defects after the release of the product or application.

4. **Regression Testing:** Regression Testing is software testing that ensures that the newly made changes to the code or newly developed modules or functions should not affect the normal processing of the application or product.
5. **System Integration Testing:** System Integration Testing is a type of testing technique that will be used to check the entire component or modules of the system in a single run. It ensures that the whole system will be checked in a single test environment instead of checking each module or function separately.

Test Execution Process

The test Execution technique consists of three different phases which will be carried out to process the test result and ensure the correctness of the required results. In each phase, various activities or work will be carried out by various team members. The three main phases of test execution are the creation of test cases, test case execution, and validation of test results. Let us discuss each phase.

1. Creation of Test Cases: The first phase is to create suitable test cases for each module or function. Here, the tester with good domain knowledge must be required to create suitable test cases. It is always preferable to create simple test cases and the creation of test cases should not be delayed else it will cause excess time to release the product. The created test cases should not be repeated again. It should cover all the possible scenarios raised in the application.

2. Test Cases Execution: After test cases have been created, execution of test cases will take place. Here, the Quality Analyst team will either do automated or manual testing depending upon the test case scenario. It is always preferable to do both automated as well as manual testing to have 100% assurance of correctness. The selection of testing tools is also important to execute the test cases.

3. Validating Test Results: After executing the test cases, note down the results of each test case in a separate file or report. Check whether the executed test cases achieved the expected result and record the time required to complete each test case i.e., measure the performance of each test case. If any of the test cases is failed or not satisfied the condition then report it to the development team for validating the code.

Ways to Perform Test Execution

Testers can choose from the below list of preferred methods to carry out test execution:

1. **Run test cases:** It is a simple and easiest approach to run test cases on the local machine and it can be coupled with other artifacts like test plans, test suites, test environments, etc.

2. **Run test suites:** A test suite is a collection of manual and automated test cases and the test cases can be executed sequentially or in parallel. Sequential execution is useful in cases where the result of the last test case depends on the success of the current test case.
3. **Run test case execution and test suite execution records:** Recording test case execution and test suite execution is a key activity in the test process and helps to reduce errors, making the testing process more efficient.
4. **Generate test results without execution:** Generating test results from non-executed test cases can be helpful in achieving comprehensive test coverage.
5. **Modify execution variables:** Execution variables can be modified in the test scripts for particular test runs.
6. **Run automated and manual tests:** Test execution can be done manually or can be automated.
7. **Schedule test artifacts:** Test artifacts include video, screenshots, data reports, etc. These are very helpful as they document the results of the past test execution and provide information about what needs to be done in future test execution.
8. **Defect tracking:** Without defect tracking test execution is not possible, as during testing one should be able to track the defects and identify what when wrong and where.

After understanding different ways to perform test execution, it's worth considering how cloud-based testing platforms like **LambdaTest** can seamlessly integrate into your test execution workflow, providing much-needed scalability and reliability. LambdaTest is an AI-powered test orchestration and execution platform that offers both manual and automated testing at scale. It enhances efficiency and effectiveness in executing test cases and suites, offering a seamless and more productive experience.

Test Execution Priorities

Test Execution Priorities are nothing but prioritizing the test cases depending upon several factors. It means that it executes the test cases with high efficient first than the other test cases. It depends upon various factors. Let us discuss some of the factors to be considered while prioritizing the test cases.

- **Complexity:** The complexity of the test cases can be determined by including several factors such as boundary values of test cases, features or components of test cases, data entry of test cases, and how much the test cases cover the given business problem.
- **Risk Covered:** How much risk that a certain test case may undergo to achieve the result. Risk in the form of time required to complete the test case process, space complexity whether it is executed in the given memory space, etc.,
- **Platforms Covered:** It simply tells that in which platform or operating system the test cases have been executed i.e., test cases executed in the Windows OS, Mac OS, Mobile OS, etc.,

- **Depth:** It covers how depth the given test cases cover each functionality or module in the application i.e., how much a given test procedure covers all the possible conditions in a single functionality or module.
- **Breadth:** It covers how the breadth of the given test cases covers the entire functionality or modules in the application i.e., how much a given test procedure covers all the possible conditions in the entire functionality or modules in the product or application.

Test Execution States

The tester or the Quality Analyst team reports or notices the result of each test case and records it in their documentation or file. There are various results raised when executing the test cases. They are

- **Pass:** It tells that the test cases executed for the module or function are successful.
- **Fail:** It tells that the test cases executed for the module or function are not successful and resulted in different outputs.
- **Not Run:** It tells that the test cases are yet to be executed.
- **Partially Executed:** It tells that only a certain number of test cases are passed and others aren't met the given requirement.
- **Inconclusive:** It tells that the test cases are executed but it requires further analysis before the final submission.
- **In Progress:** It tells that the test cases are currently executed.
- **Unexpected Result:** It tells that all the test cases are executed successfully but provide different unexpected results.

Scaffolding

Code developed to facilitate testing is called scaffolding, by analogy to the temporary structures erected around a building during construction or maintenance. Scaffoldings may include

- Test drivers (substituting for a main or calling population)
- Test harness (substituting for parts of the deployment environment)
- Stubs (substituting for functionally called or used by the software under test)

The purpose of scaffolding is to provide controllability to execute test cases and observability to judge the outcome of test execution.

Sometimes scaffolding is required to simply make module executable, but even in incremental development with immediate integration of each module, scaffolding for controllability and observability may be required because the external interfaces of the system may not provide sufficient control to drive the module under test through test cases, or sufficient observability of the effect.

GENERIC VERSUS SPECIFIC SCAFFOLDING

How general should scaffolding be? To answer We could build a driver and stubs for each test case or at least factor out some common code of the driver and test management (e.g., JUnit) ... or further factor out some common support code, to drive a large number of test cases from data... or further, generate the data automatically from a more abstract model (e.g., network trafficmodel)
Fully generic scaffolding may suffice for small numbers of hand-written test cases

The simplest form of scaffolding is a driver program that runs a single, specific test case.
It is worthwhile to write more generic test drivers that essentially interpret test case specifications.
A large suite of automatically generated test cases and a smaller set of handwritten test cases can share the
same underlying generic test scaffolding

Scaffolding to replace portions of the system is somewhat more demanding and again both generic and
application-specific approaches are possible

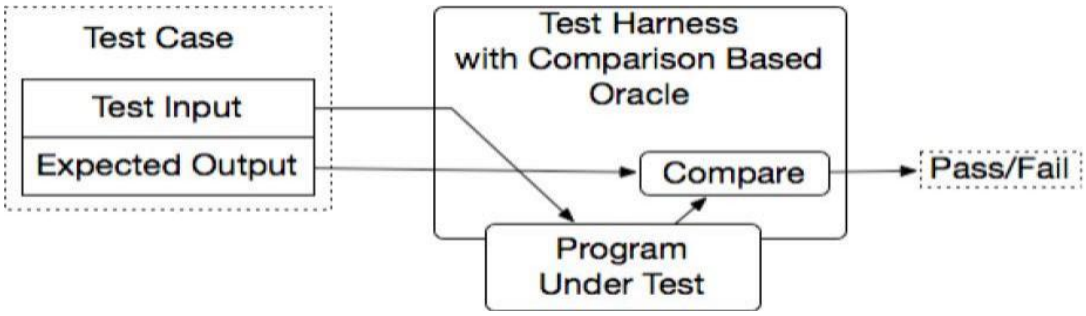
A simplest stub – ***mock*** – can be generated automatically by analysis of the source code
The balance of quality, scope and cost for a substantial piece of scaffolding software can be used in several projects

The balance is altered in favour of simplicity and quick construction for the many small pieces of scaffolding that are typically produced during development to support unit and small-scale integration testing A question of costs and re-use – Just as for other kinds of software

TEST ORACLES

In practice, the pass/fail criterion is usually imperfect. A test oracle may apply a pass/fail criterion that reflects only a part of the actual program specification, or is an approximation, and therefore passes some program executions it ought to fail Several partial test oracles may be more cost-effective than one that is more comprehensive
A test oracle may also give false alarms, failing an execution that is ought to pass. False alarms in test execution are highly undesirable. The best oracle we can obtain is an oracle that detects deviations from expectation that may or may not be actual failure.

Two types



❖ **Comparison based oracle**

Fig: a test harness with a comparison based test oracle processes test cases consisting of (program input, predicted output) pairs.

o With a comparison based oracle , we need predicted output for each input

o Oracle compares actual to predicted output, and reports failure if they differ.

o It is best suited for small number of hand generated test cases example: for handwritten Junit test cases.

o They are used mainly for small, simple test cases

o Expected outputs can also be produced for complex test cases and large test suites

o Capture-replay testing, a special case in which the predicted output or behavior is preserved from an earlier execution

o Often possible to judge output or behavior without predicting it

SELF-CHECKS AS ORACLES

An oracle can also be written as self checks

-Often possible to judge correctness without predicting results. Typically these self checks are in the form of assertions, but designed to be checked during execution.

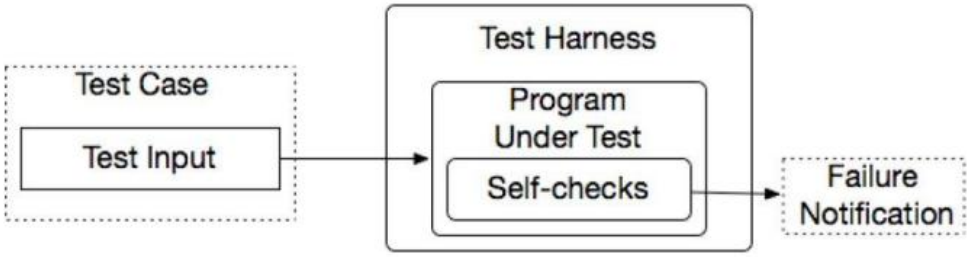
It is generally considered good design practice to make assertions and self checks to be free of side effects on program state.

Self checks in the form of assertions embedded in program code are useful primarily for checking module and subsystem-level specification rather than all program behaviour.

Devising the program assertions that correspond in a natural way to specifications poses two main challenges:

Bridging the gap between concrete execution values and abstractions used in specification

Dealing in a reasonable way with quantification over collection of values



Structural invariants are good candidates for self checks implemented as assertions
They pertain directly to the concrete data structure implementation It is sometimes straight-forward to translate quantification in a specification statement into iteration in a program assertion A run time assertion system must manage ghost variables They must retain “before” values
They must ensure that they have no side effects outside assertion checking

Advantages:
-Usable with large, automatically generated test suites.
Limits:
-often it is only a partial check.
-recognizes many or most failures, but not all.

Capture and reply

Sometimes it is difficult to either devise a precise description of expected behaviour or adequately characterize correct behaviour for effective self checks.

Example: even if we separate testing program functionally from GUI, some testing of the GUI is required. If one cannot completely avoid human involvement test case execution, one can at least avoid unnecessary repetition of this cost and opportunity for error.

The principle is simple:
The first time such a test case is executed, the oracle function is carried out by a human, and the interaction sequence is captured. Provided the execution was judged (by human tester) to be correct, the captured log now forms an (input, predicted output) pair for subsequent automated testing. The savings from automated retesting with a captured log depends on how many build-and-test cycles we can continue to use it, before it is invalidated by some change to theprogram. Mapping from concrete state to an abstract model of interacting sequences is some time possible but is generally quite limited.